

GUIDA TECNICA · FAI DA TE

Il flusso commento e messaggio su Instagram, passo per passo

Una parola nei commenti e la persona riceve un messaggio privato con la tua risorsa. Qui ci sono tutti i passaggi per costruirlo da solo con le API ufficiali di Meta: l'app, il ricevitore, il flusso, il codice completo e le prove da fare prima di accendere.

Commento con la parola → **Risposta pubblica** → **Messaggio con pulsante** → **Follow** → **Link e domanda**

Cosa ti serve

- Un account Instagram **professionale** (Business o Creator).
- Un **secondo account** Instagram, solo per le prove.
- Un account su **developers.facebook.com**.
- **Node.js 18** o più. Il codice è in JavaScript, la logica vale in qualsiasi linguaggio.
- Un indirizzo **https pubblico** per il ricevitore. Per le prove basta un tunnel (ngrok, Cloudflare Tunnel o simili), per la produzione un servizio sempre acceso.

Come leggerla

- Ogni passo ha una riga **CONTROLLO** : ti dice come capire che è andato a buon fine, prima di passare al successivo.
- I nomi dei menu del pannello Meta cambiano spesso: se non trovi una voce, cerca il nome in inglese tra le impostazioni dell'app.
- Le chiamate hanno **<SEGNAPOSTO>** da sostituire. Token e App Secret sono segreti: mai nel codice pubblicato.

LA MAPPA: 14 PASSI IN 6 PARTI

- A · **A · L'app su Meta** passi 1-5: app, permessi, token, account di prova
- B · **B · Il ricevitore** passi 6-8: l'indirizzo che Meta chiama a ogni commento
- C · **C · Il flusso** passi 9-14: le chiamate, una per una
- D · **D · Il codice completo** un file solo, collaudato
- E · **E · Le prove** cosa controllare e cosa fare se non funziona
- F · **F · Regole e produzione** limiti di Meta, policy, rinnovo del token

PARTE A · L'APP SU META

1 Crea l'app

Vai su **developers.facebook.com**, accedi e scegli **My Apps → Create App**. Come caso d'uso scegli quello per gestire messaggi e contenuti su Instagram e, se te lo chiede, il tipo **Business**. Un'app che non è di tipo Business non va bene: se ne hai una vecchia, creane una nuova.

CONTROLLO vedi il pannello dell'app, con il menu **Instagram** a sinistra.

2 Apri la configurazione di Instagram

Nel menu a sinistra: **Instagram → API setup with Instagram business login**. Meta la chiama anche Instagram API with Instagram Login: è la strada più semplice, non richiede una Pagina Facebook. Qui trovi account, token e webhook.

3 Controlla i permessi

Te ne servono tre:

```
instagram_business_basic
instagram_business_manage_comments
instagram_business_manage_messages
```

Per il tuo account, e per chi ha un ruolo nell'app, **non serve la revisione di Meta**: basta l'accesso Standard. La revisione serve solo se usi l'app per account di altri.

4 Genera il token e prendi gli ID

Nella stessa pagina, accanto al tuo account, clicca **Generate token**, accedi a Instagram e copia il token: dura **60 giorni**. Poi vai su **Impostazioni app → Di base** e copia l'**App Secret** (clic su Mostra). Infine chiedi a Meta l'ID del tuo account:

```
GET https://graph.instagram.com/v25.0/me?fields=user_id,username&access_token=<TOKEN>
```

CONTROLLO la risposta contiene il tuo username e un **user_id**: è il tuo **IG_ID**, lo stesso che troverai come **id** negli avvisi dei webhook. Segna tre valori: **TOKEN**, **APP_SECRET**, **IG_ID**.

5 Aggiungi il secondo account come tester

Nel pannello: **App roles → Roles → Add people → Instagram Tester**, e scrivi lo username del secondo account. Poi, **dal secondo account**, in Instagram: **Impostazioni → App e siti web → Inviti tester** e accetta.

Perché serve: con l'accesso Standard l'app vede solo le persone che hanno un ruolo nell'app. Senza questo passo i commenti del secondo account non arrivano, e penserai che il resto sia sbagliato.

CONTROLLO nel pannello il secondo account risulta tra i tester, con invito accettato.

PARTE B · IL RICEVITORE

6 Metti online il ricevitore

Il codice è nella parte D. Avvialo e rendilo raggiungibile da un indirizzo **https** pubblico: per le prove un tunnel verso la porta 3000, in produzione un servizio sempre acceso. Le variabili da impostare sono in fondo alla parte D.

CONTROLLO aprendo l'indirizzo dal browser il server risponde (anche solo «no»): vuol dire che è raggiungibile.

7 Registra il webhook

Nella pagina del passo 2 cerca **Configure webhooks** e compila:

- **Callback URL**: l'indirizzo https del ricevitore.
- **Verify token**: una stringa a tua scelta, la stessa che metti in **VERIFY_TOKEN**.

Clicca **Verify and save**. Meta manda al tuo indirizzo una richiesta GET con **hub.mode**, **hub.verify_token** e **hub.challenge**: il server deve rispondere con il valore di **hub.challenge**. Il codice lo fa già.

CONTROLLO il pannello salva senza errori.

8 Iscrivi il tuo account agli eventi

```
POST https://graph.instagram.com/v25.0/me/subscribed_apps
     ?subscribed_fields=comments,messages,messaging_postbacks
     &access_token=<TOKEN>
```

Nel pannello, nella stessa sezione dei webhook, attiva anche i campi **comments**, **messages** e **messaging_postbacks**.

CONTROLLO la risposta è `{"success": true}`. Con `GET .../me/subscribed_apps` vedi i campi iscritti.

Le 4 regole del ricevitore, secondo Meta

- Rispondi **200 subito** a ogni avviso, poi lavora. Se non rispondi, Meta riprova per **36 ore**.
- **Scarta i doppioni**: per via dei tentativi lo stesso avviso può arrivare più volte.
- **Verifica la firma**: l'intestazione **X-Hub-Signature-256** è un HMAC SHA256 del corpo **grezzo**, calcolato con l'App Secret. Se non coincide, scarta.
- Gli avvisi arrivano **a gruppi** (fino a 1000): scorri ogni elemento, non guardare solo il primo.

PARTE C · IL FLUSSO, CHIAMATA PER CHIAMATA

9 Arriva il commento

Meta manda al ricevitore un avviso di tipo **comments**. Ti servono quattro valori: l'**id del commento**, l'**id di chi ha commentato**, il **testo** e l'**id del post**.

```
{ "object": "instagram", "entry": [ { "id": "<IG_ID>", "time": 1700000000,
  "changes": [ { "field": "comments", "value": {
    "id": "<COMMENT_ID>",
    "from": { "id": "<ID_PERSONA>", "username": "nome" },
    "text": "voglio la GUIDA",
    "media": { "id": "<MEDIA_ID>", "media_product_type": "REELS" } } } ] } ] }
```

Controlla se il testo contiene la tua parola (parola intera, senza distinguere maiuscole) e gestisci **solo il primo commento** di ogni persona su quel post.

10 Risposta pubblica, sotto il commento

```
POST https://graph.instagram.com/v25.0/<COMMENT_ID>/replies
Authorization: Bearer <TOKEN>
{ "message": "Fatto, guarda nei messaggi" }
```

Funziona solo sui commenti di primo livello. Alterna 3-5 varianti.

11 Risposta privata: il primo messaggio

Si manda con l'**id del commento** al posto dell'id della persona. Meta ne consente **uno solo per commento**, entro **7 giorni**. Arriva nella posta in arrivo se la persona ti segue, nella cartella **Richieste** se non ti segue, con il link al post.

Prova 1 · con il pulsante (è quello che vuoi: il tocco apre le 24 ore):

```
POST https://graph.instagram.com/v25.0/<IG_ID>/messages
Authorization: Bearer <TOKEN>
{ "recipient": { "comment_id": "<COMMENT_ID>" },
  "message": { "attachment": { "type": "template", "payload": {
    "template_type": "button",
    "text": "Ciao, eccomi. Ti mando subito la guida. Tocca il pulsante e arriva.",
    "buttons": [ { "type": "postback", "title": "Mandamela", "payload": "LINK:GUIDA" } ] } } } }
```

> **Da provare, non è garantito.** Nella documentazione di Meta la risposta privata è descritta solo con testo semplice, e il pulsante solo con l'id della persona al posto dell'id del commento. Che si possano unire non è scritto. Il codice della parte D prova il pulsante e, se Meta lo rifiuta, passa da solo alla prova 2.

Prova 2 · solo testo, e la persona risponde (funziona sempre):

```
{ "recipient": { "comment_id": "<COMMENT_ID>" },
  "message": { "text": "Ciao, eccomi. Ti mando subito la guida. Rispondi con la parola VIA e arriva." } }
```

Quando la persona risponde, nel ricevitore arriva un avviso **messages** e le 24 ore sono aperte, come con il tocco.

CONTROLLO dal secondo account il messaggio arriva (guarda anche **Richieste**). Tieni nota di quale prova ha funzionato: è l'informazione più importante di tutta la guida.

12 Il tocco sul pulsante

Quando la persona tocca, arriva un avviso **messaging_postbacks**. Da qui hai il suo **id** (**sender.id**) e **24 ore** per scrivere.

```
{ "object": "instagram", "entry": [ { "id": "<IG_ID>", "messaging": [ {
  "sender": { "id": "<ID_PERSONA>" }, "recipient": { "id": "<IG_ID>" },
  "postback": { "mid": "...", "title": "Mandamela", "payload": "LINK:GUIDA" } } ] } ] }
```

Il **payload** è la stringa che hai scelto tu: qui ti dice che parola sta aspettando.

13 Il follow

Prima del link, chiedi di seguirti e controlla se lo fa. Il controllo è una chiamata sull'id della persona:

```
GET https://graph.instagram.com/v25.0/<ID_PERSONA>?fields=is_user_follow_business
Authorization: Bearer <TOKEN>      →  { "is_user_follow_business": false }
```

Funziona solo dopo che la persona ha toccato un pulsante o risposto. Il modo che consiglio è **morbido**: se non ti segue, mandi il messaggio con la richiesta e un secondo pulsante; **al secondo tocco il link parte comunque**. Il controllo a volte sbaglia sui follower di vecchia data, e bloccare un follower vero costa più di un link regalato.

14 Il link e la domanda

```
POST https://graph.instagram.com/v25.0/<IG_ID>/messages
Authorization: Bearer <TOKEN>
{ "recipient": { "id": "<ID_PERSONA>" },
  "message": { "text": "Ecco la guida: https://...\n\nQuale parte vuoi mettere in pratica per prima?" } }
```

Il link inizia con **https**. La domanda è una sola e si risponde in una riga: è lei che fa partire la conversazione.

CONTROLLO dal secondo account: commento, risposta pubblica, messaggio, tocco, richiesta di follow, secondo tocco, link. Se tutta la catena torna, il flusso funziona.

PARTE D · IL CODICE COMPLETO (1 DI 2)

Un file solo, **bot.mjs**, senza librerie da installare. Fa tutta la catena: verifica, firma, doppioni, risposta pubblica, prova 1 e prova 2, follow morbido, link. Le regole (una riga per parola) sono all'inizio: cambiale con le tue.

```
import http from "node:http";
import crypto from "node:crypto";

const { VERIFY_TOKEN, APP_SECRET, IG_ID, IG_TOKEN, PORT = 3000 } = process.env;
const API = process.env.GRAPH || "https://graph.instagram.com/v25.0";

// 1. Le regole: una riga per parola
const RULES = {
  GUIDA: {
    attiva: true,
    risposte: ["Ti ho scritto in privato", "Fatto, guarda nei messaggi", "Arrivato, controlla i messaggi"],
    dm: "Ciao, eccomi. Ti mando subito la guida. Tocca il pulsante e arriva. (Messaggio automatico)",
    pulsante: "Mandamela",
    follow: "Ci sei quasi! Seguimi, così vedi anche i prossimi video, poi ritocca il pulsante.",
    link: "https://esempio.it/guida",
    domanda: "Quale parte vuoi mettere in pratica per prima?",
  },
};

const visti = new Set();        // deduplica: commenti già gestiti
const chiesto = new Set();      // chi ha già ricevuto la richiesta di follow

const post = (path, body) =>
  fetch(`${API}${path}`, {
    method: "POST",
    headers: { "Content-Type": "application/json", Authorization: `Bearer ${IG_TOKEN}` },
    body: JSON.stringify(body),
  }).then(async (r) => ({ ok: r.ok, status: r.status, json: await r.json().catch(() => ({})) }));

const pulsante = (testo, titolo, payload) => ({
  attachment: {
    type: "template",
    payload: { template_type: "button", text: testo, buttons: [{ type: "postback", title: titolo, payload }] },
  },
});

const invia = (igsid, message) => post(`/${IG_ID}/messages`, { recipient: { id: igsid }, message });

function trovaParola(testo) {
  const parole = (testo || "").toUpperCase().split(/[^A-ZÀ-Ý0-9]+/);
  return Object.keys(RULES).find((p) => RULES[p].attiva && parole.includes(p));
}

// 2. Arriva un commento
async function gestisciCommento(c) {
  if (visti.has(c.id) || c.from.id === IG_ID) return;
  visti.add(c.id);
  const parola = trovaParola(c.text);
  if (!parola) return;
  const chiave = `${c.from.id}:${c.media.id}`;           // solo il primo commento di ogni persona
  if (visti.has(chiave)) return;
  visti.add(chiave);
  const r = RULES[parola];

  await post(`/${c.id}/replies`, { message: r.risposte[Math.floor(Math.random() * r.risposte.length)] });

  // PROVA 1: risposta privata con il pulsante
  let esito = await post(`/${IG_ID}/messages`, {
    recipient: { comment_id: c.id },
    message: pulsante(r.dm, r.pulsante, `LINK:${parola}`),
  });
  // PROVA 2: se Meta rifiuta il pulsante, testo semplice che chiede di rispondere
  if (!esito.ok) {
    console.log("pulsante rifiutato, uso il testo:", esito.status, JSON.stringify(esito.json));
    await post(`/${IG_ID}/messages`, {
      recipient: { comment_id: c.id },
      message: { text: `${r.dm.split(" Tocca")[0]} Rispondi a questo messaggio con la parola VIA e te la mando.` },
    });
  }
}
```

```
// 3. Arriva un tocco sul pulsante (o la risposta di testo della PROVA 2)
async function gestisciMessaggio(m) {
  const igsid = m.sender.id;
  if (igsid === IG_ID || m.message?.is_echo) return;
  let parola, secondoTocco = false;
  if (m.postback) {
    const [tipo, p, n] = m.postback.payload.split(":");
    parola = p; secondoTocco = n === "2";
    if (tipo !== "LINK") return;
  } else if (m.message?.text && /^\s*via\s*$/i.test(m.message.text)) {
    parola = Object.keys(RULES)[0];
  } else return;
  const r = RULES[parola];
  if (!r) return;

  // 4. Il follow: controllo morbido, al secondo tocco il link parte comunque
  const prof = await fetch(`${API}/${igsid}?fields=is_user_follow_business`, {
    headers: { Authorization: `Bearer ${IG_TOKEN}` },
  }).then((x) => x.json()).catch(() => ({}));
  if (!prof.is_user_follow_business && !secondoTocco && !chiesto.has(igsid)) {
    chiesto.add(igsid);
    await invia(igsid, pulsante(r.follow, r.pulsante, `LINK:${parola}:2`));
    return;
  }
  // 5. Il link e la domanda
  await invia(igsid, { text: `Ecco la guida: ${r.link}\n\n${r.domanda}` });
}

// Il ricevitore
http.createServer(async (req, res) => {
  const url = new URL(req.url, "http://x");
  if (req.method === "GET") {                                  // verifica di Meta
    const ok = url.searchParams.get("hub.mode") === "subscribe" &&
      url.searchParams.get("hub.verify_token") === VERIFY_TOKEN;
    res.writeHead(ok ? 200 : 403).end(ok ? url.searchParams.get("hub.challenge") : "no");
    return;
  }
  const chunks = []; for await (const c of req) chunks.push(c);
  const raw = Buffer.concat(chunks);
  const atteso = "sha256=" + crypto.createHmac("sha256", APP_SECRET).update(raw).digest("hex");
  const arrivato = req.headers["x-hub-signature-256"] || "";
  const valida = arrivato.length === atteso.length &&
    crypto.timingSafeEqual(Buffer.from(arrivato), Buffer.from(atteso));
  if (!valida) { res.writeHead(401).end(); return; }
  res.writeHead(200).end("ok");                                // rispondi subito, poi lavora
  try {
    for (const e of JSON.parse(raw).entry || []) {
      for (const ch of e.changes || []) if (ch.field === "comments") await gestisciCommento(ch.value);
      for (const m of e.messaging || []) await gestisciMessaggio(m);
    }
  } catch (err) { console.error(err); }
}).listen(PORT, () => console.log("in ascolto su", PORT));
```

Come avviarlo

```
export VERIFY_TOKEN="la-stringa-del-passo-7"
export APP_SECRET="l-app-secret-del-passo-4"
export IG_ID="l-id-del-passo-4"
export IG_TOKEN="il-token-del-passo-4"
node bot.mjs        # in ascolto sulla porta 3000
```

Cosa è stato verificato: la logica (firma, doppioni, parola intera, sequenza dei passi, ripiego se il pulsante viene rifiutato) è collaudata con un finto server Meta, 17 controlli su 17. **Non è stato collaudato contro Meta:** quello lo fai tu con la parte E.

Limiti di questo file: i doppioni e chi ha già ricevuto la richiesta di follow stanno in memoria, e si perdono se riavvii. Per l'uso vero salvali in un file o in un database.

PARTE E · LE PROVE, PRIMA DI ACCENDERE

1. Pubblica (o usa) un post di prova sul tuo account e scrivi nella didascalia la parola.
2. Dal **secondo account** commenta con la parola. Poi guarda i log del ricevitore.
3. Controlla: risposta pubblica sotto il commento, messaggio privato (anche in **Richieste**), tocco o risposta, richiesta di follow, secondo tocco, link.
4. Annota **quale prova** del passo 11 ha funzionato (pulsante o solo testo) e adatta i testi di conseguenza.
5. Apri il link **dal telefono**: deve iniziare con https e caricarsi.
6. Commenta due volte dallo stesso account: la seconda volta non deve succedere niente.
7. La parola resta **spenta** (`attiva: false`) fino al giorno in cui pubblichi.

SE NON FUNZIONA

COSA VEDI	DA CONTROLLARE
Il pannello non salva il webhook	L'indirizzo non è https pubblico, il server non risponde con **hub.challenge**, o il **Verify token** è diverso da VERIFY_TOKEN.
Il webhook è salvato, ma non arriva niente	Account non iscritto (passo 8), campi non attivi, il commentatore **non è tester** (passo 5), account non professionale, commento su un post di un altro account.
Arriva, ma il bot risponde 401	La firma. Va calcolata sul corpo **grezzo**, non sul JSON riletto, e con l'App Secret di **questa** app.
Errore sulla risposta privata	Commento più vecchio di 7 giorni, messaggio già mandato per quel commento, token scaduto o permesso mancante. Leggi il codice dell'errore nella risposta di Meta.
Il pulsante viene rifiutato	È il caso della prova 2: il codice passa al testo. Controlla nei log il messaggio «pulsante rifiutato».
Il messaggio non si vede	Cerca nella cartella **Richieste**: se il secondo account non ti segue, ci finisce lì.
Messaggi doppi	Manca la deduplica. Meta riprova se la tua risposta tarda: rispondi 200 prima di lavorare.
Dopo qualche settimana smette	Il token dura 60 giorni. Rinnovalo prima della scadenza (parte F).

I limiti di Meta

- Un solo messaggio automatico per commento, entro 7 giorni.
- Per scrivere ancora serve una risposta o un tocco: da lì hai 24 ore.
- Fino a **750 risposte private all'ora** per account, sui commenti dei post e dei reel.
- Il messaggio non può partire se il commento è nascosto, o di una diretta già finita.
- Si risponde in pubblico solo ai commenti di primo livello.

Le buone abitudini

- **Dillo che è automatico** nel primo messaggio. Meta lo richiede dove la legge lo impone e lo consiglia sempre.
- Una regola sola per parola, e la parola uguale nel video e nella didascalia.
- Salva solo ciò che ti serve (id, parola, esito) e di' nella tua informativa cosa conservi.
- Niente invii a chi non ti ha scritto.

PER TENERLO ACCESO DAVVERO

1. **Servizio sempre acceso** per il ricevitore, con un riavvio automatico se cade.
2. **Archivio** al posto della memoria: commenti già gestiti e chi ha ricevuto la richiesta di follow.
3. **Rinnova il token** prima dei 60 giorni (deve avere almeno 24 ore e non essere scaduto; dopo il rinnovo vale altri 60 giorni):

```
GET https://graph.instagram.com/refresh_access_token
    ?grant_type=ig_refresh_token&access_token=<TOKEN>
```

4. **Un registro** di chi ha chiesto cosa e se il link è arrivato: ti dice quale video porta contatti.
5. Se vuoi usarlo per **account di altri** (clienti), serve la revisione di Meta: è un lavoro a parte, con video e permessi uno per uno.

Onestà, cosa questa guida non dice

- **Le regole e le API di Meta cambiano.** I passaggi sono controllati sulla documentazione ufficiale il 29/10/2026 (pagine aggiornate tra marzo e giugno 2026): ricontrolla prima di costruire.
- **Il pulsante dentro la risposta privata non è documentato da Meta.** Per questo ci sono due prove. Non so dirti in anticipo quale passerà sul tuo account.
- **I nomi dei menu** del pannello Meta cambiano: quelli indicati sono la mia lettura della documentazione, non una schermata di oggi.
- **Il follow:** chiedere di seguire prima della risorsa non è vietato, ma Meta non lo approva espressamente. Valuta tu.
- **Settori regolati**, come la sanità: Meta limita l'uso dei messaggi per mettere in contatto persone e strutture sanitarie. Leggi le sue policy prima.
- Copre **un percorso solo**, dal commento alla risorsa. Flussi più lunghi o invii a una lista non sono qui.

Ti blocchi su un passo? Ti aiuto a montarlo.

Se l'app, il ricevitore o le prove non ti tornano, **prenota una call di 15 minuti** e lo guardiamo insieme · **automis.ai**